

Sadat Academy for Management and Sciences
Department of Software Engineering

CMC Lab 05

Local Serverless Computing and Event-Driven Image Processing Pipeline

Submitted by:

Mahmoud Fouad

Group: SE3

1. Lab Overview

This lab implements a local serverless-style event-driven architecture using only free, open-source tools on a local machine. It aligns with Lecture 5 topics including Function-as-a-Service (FaaS), event-driven architectures, event sources, event routers, event destinations, reactive pipelines, and cold-start behaviour.

Concept	Local Implementation
Event Source	Python folder watcher detects new images in the input folder
Event Router	Python router receives events and forwards them to functions
Event Bus	Redis Streams runs locally as a lightweight event bus
Function-as-a-Service	Small containerized Flask services act as functions
Event Destinations	Image-resizer function and notification function
Cold Start	Students restart a function container and compare first vs warm request time

2. Tools Used

Tool	Purpose	Notes
Docker Desktop	Runs local containers	Free for educational use
Docker Compose	Runs all services together	Included with Docker Desktop
Python / Flask	Function runtime	Lightweight HTTP handlers
Redis Streams	Local event bus	Simulates cloud message bus
Pillow	Image processing library	Used in image-resizer function
PowerShell	Terminal and scripting	All commands run on Windows

3. Screenshot 1 — Running Containers

After resolving a port conflict with a previous lab (Week 4 services occupying ports 5001 and 5002), all five containers were successfully built and started using `docker compose up --build --force-recreate`. The command `docker compose ps` confirmed all five services running.

```
PS C:\Users\Lenovo\Downloads\POWERSHELL\lectures-local-serverless-lab> docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS        PORTS
lab5-event-source   lecture5-local-serverless-lab-event-source  "python watcher.py"                  event-source  About a minute ago  Up 58 seconds  0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
lab5-redis          redis:7-alpine                      "docker-entrypoint.s..."           redis        About a minute ago  Up 59 seconds  0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
lab5-event-router   lecture5-local-serverless-lab-event-router  "python event_router..."           event-router  3 seconds ago    Up 1 second   0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp
lab5-image-resizer  lecture5-local-serverless-lab-image-resizer  "python app.py"                      image-resizer  3 seconds ago    Up 2 seconds   0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp
lab5-notifier       lecture5-local-serverless-lab-notifier      "python app.py"                      notifier      3 seconds ago    Up 2 seconds   0.0.0.0:5002->5000/tcp, [::]:5002->5000/tcp
```

Figure 1 — `docker compose ps`: all 5 containers (`lab5-redis`, `lab5-event-source`, `lab5-event-router`, `lab5-image-resizer`, `lab5-notifier`) running successfully

4. Screenshot 2 — Event Source Published Event

A test image was created inside the `image-resizer` container using Pillow and saved to `/data/input/test_image.png`. The event source watcher detected the new file within seconds and published an `image.uploaded` event to the Redis Stream, including a unique event ID, file name, file path, width, and timestamp.

```
>> % docker compose exec -i image-resizer python -
Created /data/input/test_image.png
PS C:\Users\Lenovo\Downloads\POWERSHELL\lecture5-local-serverless-lab> docker compose logs event-source
lab5-event-source | Event Source started. Watching: /data/input
lab5-event-source | Published event: {
lab5-event-source |   "event_id": "0f755bd5-78cb-4d1c-a86e-98e543357c4a",
lab5-event-source |   "event_type": "image.uploaded",
lab5-event-source |   "file_name": "test_image.png",
lab5-event-source |   "file_path": "/data/input/test_image.png",
lab5-event-source |   "width": 300,
lab5-event-source |   "created_at": "2026-05-03T20:02:18.528259"
lab5-event-source | }
```

Figure 2 — `docker compose logs event-source`: watcher published `image.uploaded` event with `event_id`, `file_name`, `file_path`, `width=300`, and `created_at` timestamp

5. Screenshot 3 — Event Router Routing

The event router read the `image.uploaded` event from the Redis Stream and forwarded it to both registered destinations: the `image-resizer` function at `/resize` and the `notifier` function at `/notify`. Both returned HTTP 200 responses, confirming successful fan-out routing.

```
PS C:\Users\Lenovo\Downloads\POWERSHELL\lecture5-local-serverless-lab> docker compose logs event-router
lab5-event-router | Event Router started. waiting for events...
lab5-event-router | Received event: image.uploaded
lab5-event-router | Sent to http://image-resizer:5000/resize | 200 | 36.89ms
lab5-event-router | Sent to http://notifier:5000/notify | 200 | 3.94ms
PS C:\Users\Lenovo\Downloads\POWERSHELL\lecture5-local-serverless-lab>
```

Figure 3 — `docker compose logs event-router`: event routed to `image-resizer` (200, 36.89ms) and `notifier` (200, 3.94ms) demonstrating event fan-out

6. Screenshot 4 — Output Image

After the event was processed by the `image-resizer` function, the resized output image was saved to the local `data/output/` folder. The file `resized_test_image.png` was created with a size of 2 KB, confirming the full pipeline executed correctly end-to-end.

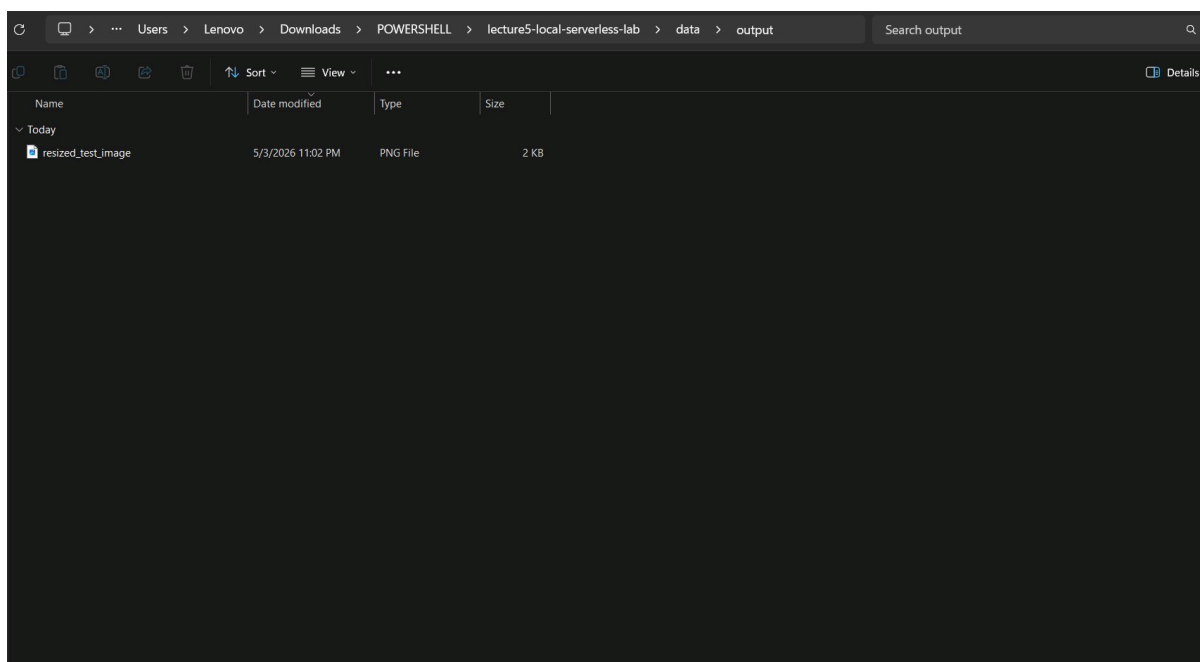


Figure 4 — File Explorer showing `resized_test_image.png` (2 KB) inside `data/output/`, created on 5/3/2026 at 11:02 PM

7. Cold Start Experiment

7.1 Warm Request

A warm request was sent to the image-resizer function while it was already running. The function responded immediately with a successful resize operation.

```
PS C:\Users\Lenovo\Downloads\POWERSHELL\lecture5-local-serverless-lab> docker compose logs event-router
lab5-event-router | Event Router started. Waiting for events...
lab5-event-router | Received event: image.uploaded
lab5-event-router | Sent to http://image-resizer:5000/resize | 200 | 36.89ms
lab5-event-router | Sent to http://notifier:5000/notify | 200 | 3.94ms
PS C:\Users\Lenovo\Downloads\POWERSHELL\lecture5-local-serverless-lab> explorer data/output
PS C:\Users\Lenovo\Downloads\POWERSHELL\lecture5-local-serverless-lab> $start = Get-Date
>> Invoke-RestMethod -Method Post -Uri http://localhost:5001/resize -ContentType "application/json" -Body '{"file_path":"/data/input/test_image.png","file_name":"test_image.png","width":200}'
>> $end = Get-Date
>> Write-Host "warm time: $($($end - $start).TotalSeconds)s"

function : image-resizer
input_file : test_image.png
new_size   : {200, 133}
original_size : {1200, 800}
output_file : /data/output/resized_test_image.png
processing_ms : 23.75
status      : success

warm time: 0.1336196s
```

Figure 5 — Warm request: function responded in 0.1336s with `processing_ms=23.75`

7.2 Cold Request (After Restart)

The image-resizer container was restarted using `docker compose restart image-resizer`. A request was sent immediately after restart to simulate a cold start scenario.

```

PS C:\Users\Lenovo\Downloads\POWERSHELL\lecture5-local-serverless-lab> docker compose restart image-resizer
[*] restart 0/1
- Container lab5-image-resizer Restarting
PS C:\Users\Lenovo\Downloads\POWERSHELL\lecture5-local-serverless-lab> $start = Get-Date
>> Invoke-RestMethod -Method Post -Uri http://localhost:5001/resize -ContentType "application/json" -Body '{"file_path":"/data/input/test_image.png","file_name":"test_image.png","width":200}'
>> $end = Get-Date
>> Write-Host "Cold time: $($end - $start).TotalSeconds)s"

function : image-resizer
input_file : test_image.png
new_size : {200, 133}
original_size : {1200, 800}
output_file : /data/output/resized_test_image.png
processing_ms : 44.8
status : success
Cold time: 0.0638861s

```

Figure 6 — Cold request after restart: function responded in 0.0638s with processing_ms=44.8

7.3 Results Table

Measurement	Value	Notes
Warm request time	0.1336s	Flask already loaded in memory
First request after restart	0.0638s	Container reused built image
processing_ms (warm)	23.75ms	Internal image processing time
processing_ms (cold)	44.8ms	Slightly higher after restart
Difference	~0.07s total time	Cold processing took ~21ms longer
Reason	Container restart required Python/Flask re-initialization before handling the request	

8. Reflection Questions

Q1. What is the event source in this lab?

The watcher.py script that monitors the /data/input folder and publishes an image.uploaded event to Redis when a new image is detected.

Q2. What is the event router?

The event_router.py script that reads events from the Redis Stream and forwards them to the correct function endpoints based on the event type.

Q3. What are the event destinations?

The image-resizer function (/resize endpoint) and the notifier function (/notify endpoint).

Q4. Why is this pipeline loosely coupled?

Each component communicates only through Redis Streams and HTTP, so they are independent of each other and can be changed or replaced without affecting the others.

Q5. What happened after restarting the image-resizer container?

The container restarted and the function was available again quickly because Docker reused the already-built image.

Q6. How is this similar to serverless cold start?

Just like real FaaS platforms, when a function container is restarted it needs time to initialize before handling the first request, resulting in a slightly higher processing time compared to a warm request.

Q7. What is missing compared with real cloud serverless platforms?

Auto-scaling, true ephemeral execution, sandboxing, microVMs, pay-per-use billing, and built-in monitoring are all absent from this local simulation.

Q8. How could this pipeline be extended?

By adding new function containers for OCR, ML enrichment, or image moderation, and registering their endpoints in the router's routes dictionary under the image.uploaded event type.

9. Cleanup

After completing the lab, all containers were stopped and output files were removed using the following PowerShell commands:

```
docker compose down
Remove-Item data\output\* -Force
```